

Random Forest Model Evaluation: Airline Customer Satisfaction

Goal: Build an optimized Random Forest classifier to predict airline passenger satisfaction, using a proper three-way data split and `GridSearchCV` with `PredefinedSplit` to tune hyperparameters without leaking test data into model selection. Compare the tuned Random Forest against a single Decision Tree to quantify the benefit (and the overfitting trade-off) of the ensemble approach.

Dataset: Invistico Airline customer survey, 129,880 rows, 22 columns (1 target + 21 features: demographics, travel type, and 14 service rating categories on a 0-5 scale).

1. Imports

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV, PredefinedSplit
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, ConfusionMatrixDisplay, classification_report
)

RANDOM_STATE = 42
pd.set_option('display.max_columns', None)
```

2. Load and inspect the data

First pass: check shape, data types, missing values, and class balance before doing anything else. No point tuning a model on data we haven't looked at.

```
df = pd.read_csv('Invistico_Airline.csv')
print(df.shape)
df.head()
```

(129880, 22)

	satisfaction	Customer Type	Age	Type of Travel	Class	Flight Distance	Seat comfort	Departure/Arrival time convenient	Food and drink	Gate location	Inflight wifi service	Infl entertain
0	satisfied	Loyal Customer	65	Personal Travel	Eco	265	0		0	0	2	2
1	satisfied	Loyal Customer	47	Personal Travel	Business	2464	0		0	0	3	0
2	satisfied	Loyal Customer	15	Personal Travel	Eco	2138	0		0	0	3	2
3	satisfied	Loyal Customer	60	Personal Travel	Eco	623	0		0	0	3	3
4	satisfied	Loyal Customer	70	Personal Travel	Eco	354	0		0	0	3	4

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 129880 entries, 0 to 129879
Data columns (total 22 columns):
#   Column              Non-Null Count  Dtype
---  -
0   satisfaction        129880 non-null object
1   Customer Type       129880 non-null object
2   Age                 129880 non-null int64
3   Type of Travel      129880 non-null object
4   Class               129880 non-null object
5   Flight Distance     129880 non-null int64
```

```

6  Seat comfort                129880 non-null  int64
7  Departure/Arrival time convenient 129880 non-null  int64
8  Food and drink              129880 non-null  int64
9  Gate location               129880 non-null  int64
10 Inflight wifi service       129880 non-null  int64
11 Inflight entertainment      129880 non-null  int64
12 Online support              129880 non-null  int64
13 Ease of Online booking      129880 non-null  int64
14 On-board service            129880 non-null  int64
15 Leg room service            129880 non-null  int64
16 Baggage handling            129880 non-null  int64
17 Checkin service             129880 non-null  int64
18 Cleanliness                 129880 non-null  int64
19 Online boarding             129880 non-null  int64
20 Departure Delay in Minutes  129880 non-null  int64
21 Arrival Delay in Minutes    129487 non-null  float64
dtypes: float64(1), int64(17), object(4)
memory usage: 21.8+ MB

```

```
df.isnull().sum()[df.isnull().sum() > 0]
```

```

Arrival Delay in Minutes    393
dtype: int64

```

`Arrival Delay in Minutes` has 393 missing values out of ~130k rows (0.3%). The departure delay for those same rows looks like a normal mix of on-time and delayed flights, so there's no obvious pattern explaining the gap, it just looks like missing records. Given the delay columns are heavily right-skewed (a small number of very long delays), median imputation is the safer choice here over mean, since the mean would get pulled up by outliers.

```
df['satisfaction'].value_counts(normalize=True)
```

```

satisfaction
satisfied      0.547328
dissatisfied   0.452672
Name: proportion, dtype: float64

```

The target is reasonably balanced, about 55% satisfied vs 45% dissatisfied. Not perfectly even, but not the kind of severe imbalance that would force us into resampling or class weighting. Accuracy is a defensible primary metric here, though we'll still look at precision, recall, and F1 since a hospital... well, an airline, cares differently about false positives vs false negatives depending on the business question (more on that in the final write-up).

3. Clean and prepare features

- Impute the 393 missing arrival delay values with the column median
- Encode the target as 0/1
- One-hot encode the three categorical features (`Customer Type`, `Type of Travel`, `Class`)

```

df['Arrival Delay in Minutes'] = df['Arrival Delay in Minutes'].fillna(
    df['Arrival Delay in Minutes'].median()
)

df['satisfaction'] = df['satisfaction'].map({'satisfied': 1, 'dissatisfied': 0})

categorical_cols = ['Customer Type', 'Type of Travel', 'Class']
df_encoded = pd.get_dummies(df, columns=categorical_cols, drop_first=True)

X = df_encoded.drop(columns=['satisfaction'])
y = df_encoded['satisfaction']

print(f"Final feature count: {X.shape[1]}")
list(X.columns)

```

```

Final feature count: 22
['Age',
 'Flight Distance',
 'Seat comfort',
 'Departure/Arrival time convenient',
 'Food and drink',
 'Gate location',
 'Inflight wifi service',
 'Inflight entertainment',
 'Online support',
 'Ease of Online booking',
 'On-board service',

```

```
'Leg room service',
'Baggage handling',
'Checkin service',
'Cleanliness',
'Online boarding',
'Departure Delay in Minutes',
'Arrival Delay in Minutes',
'Customer Type_disloyal Customer',
'Type of Travel_Personal Travel',
'Class_Eco',
'Class_Eco Plus']
```

4. Three-way split: Train / Validation / Test

This is the core safeguard against data leakage in this project. The test set gets carved off first and is not touched again until the very last evaluation step. Hyperparameter tuning only ever sees train and validation data.

- **Train (60%):** fit models
- **Validation (20%):** score each hyperparameter combination during the grid search
- **Test (20%):** held out completely until the final model is locked in, used once for the final report

All splits are stratified on the target so the satisfaction rate stays consistent across train, validation, and test.

```
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.4, random_state=RANDOM_STATE, stratify=y
)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.5, random_state=RANDOM_STATE, stratify=y_temp
)

print(f"Train: {X_train.shape[0]:,} rows")
print(f"Validation: {X_val.shape[0]:,} rows")
print(f"Test: {X_test.shape[0]:,} rows")
print()
print(f"Satisfaction rate (train):      {y_train.mean():.3f}")
print(f"Satisfaction rate (validation): {y_val.mean():.3f}")
print(f"Satisfaction rate (test):       {y_test.mean():.3f}")
```

```
Train: 77,928 rows
Validation: 25,976 rows
Test: 25,976 rows
```

```
Satisfaction rate (train):      0.547
Satisfaction rate (validation): 0.547
Satisfaction rate (test):       0.547
```

5. Baseline: single Decision Tree

Before tuning a Random Forest, we need a baseline to compare it against. A single Decision Tree, trained on the training set and capped at `max_depth=10` to keep it from immediately overfitting to noise, gives us a reference point for accuracy, precision, recall, and F1 as well as the train/test accuracy gap (our proxy for overfitting).

```
dt_baseline = DecisionTreeClassifier(max_depth=10, random_state=RANDOM_STATE)
dt_baseline.fit(X_train, y_train)

dt_test_pred = dt_baseline.predict(X_test)
dt_train_pred = dt_baseline.predict(X_train)

dt_metrics = {
    'Accuracy': accuracy_score(y_test, dt_test_pred),
    'Precision': precision_score(y_test, dt_test_pred),
    'Recall': recall_score(y_test, dt_test_pred),
    'F1 Score': f1_score(y_test, dt_test_pred)
}

dt_train_acc = accuracy_score(y_train, dt_train_pred)
dt_test_acc = dt_metrics['Accuracy']

print("Decision Tree baseline, test set performance:")
for k, v in dt_metrics.items():
    print(f" {k}: {v:.4f}")
print()
print(f"Train accuracy: {dt_train_acc:.4f}")
print(f"Test accuracy: {dt_test_acc:.4f}")
print(f"Train-test gap: {dt_train_acc - dt_test_acc:.4f}")
```

```
Decision Tree baseline, test set performance:
Accuracy: 0.9202
Precision: 0.9250
Recall: 0.9297
F1 Score: 0.9273

Train accuracy: 0.9286
Test accuracy: 0.9202
Train-test gap: 0.0083
```

The Decision Tree lands around 92% accuracy on the test set with a fairly small train-test gap, so it's a competent model on its own, not a strawman. That makes the comparison with Random Forest meaningful instead of a guaranteed win.

6. Hyperparameter tuning: GridSearchCV with PredefinedSplit

This is the part of the project that actually prevents data leakage during model selection.

A normal `GridSearchCV` call with `cv=5` would randomly re-split the training data internally, which is fine on its own, but it gives no control over *which* rows get used for scoring. We already have a clean, separate validation set we built by hand. `PredefinedSplit` lets us tell `GridSearchCV` to use exactly that split: train on the rows marked `-1`, evaluate only on the rows marked `0` (our validation rows). The test set never enters this step at all.

We combine train and validation into one array purely so `GridSearchCV` can iterate over it, but the fold definition guarantees the model is fit only on the training rows and scored only on the validation rows for every parameter combination.

```
X_gridsearch = pd.concat([X_train, X_val], axis=0)
y_gridsearch = pd.concat([y_train, y_val], axis=0)

# -1 marks rows that are always in the training fold (never scored)
# 0 marks rows that are always in the validation fold (always scored)
split_index = np.concatenate([
    np.full(len(X_train), -1),
    np.full(len(X_val), 0)
])
predefined_split = PredefinedSplit(test_fold=split_index)

param_grid = {
    'n_estimators': [100, 200],
    'max_depth': [10, 20, None],
    'min_samples_leaf': [1, 5]
}

rf = RandomForestClassifier(random_state=RANDOM_STATE, n_jobs=-1)

grid_search = GridSearchCV(
    estimator=rf,
    param_grid=param_grid,
    cv=predefined_split,
    scoring='f1',
    n_jobs=-1,
    verbose=1
)

grid_search.fit(X_gridsearch, y_gridsearch)

print("\nBest parameters (by validation F1):", grid_search.best_params_)
print(f"Best validation F1 score: {grid_search.best_score_:.4f}")
```

Fitting 1 folds for each of 12 candidates, totalling 12 fits

```
Best parameters (by validation F1): {'max_depth': None, 'min_samples_leaf': 1, 'n_estimators': 100}
Best validation F1 score: 0.9580
```

```
results_df = pd.DataFrame(grid_search.cv_results_[
    ['param_n_estimators', 'param_max_depth', 'param_min_samples_leaf', 'mean_test_score', 'rank_test_score']
]).sort_values('rank_test_score').reset_index(drop=True)

results_df.columns = ['n_estimators', 'max_depth', 'min_samples_leaf', 'validation_f1', 'rank']
results_df
```

	n_estimators	max_depth	min_samples_leaf	validation_f1	rank
0	100	None	1	0.957996	1
1	200	20	1	0.957114	2
2	200	None	1	0.957093	3
3	100	20	1	0.956691	4
4	200	20	5	0.953250	5
5	200	None	5	0.953243	6
6	100	None	5	0.952878	7
7	100	20	5	0.951823	8
8	100	10	1	0.932416	9
9	200	10	1	0.931911	10
10	200	10	5	0.931323	11
11	100	10	5	0.930964	12

A few things worth flagging from this table before moving on:

- `max_depth=10` is the clear loser across every other setting, validation F1 drops by roughly 2.5 points compared to deeper trees. The Random Forest needs more depth than a single tree to actually benefit from averaging across many trees.
- `min_samples_leaf=1` edges out `min_samples_leaf=5` by a small margin (about half a point of F1). That's a real but modest difference, which matters for how we interpret the overfitting discussion in the final evaluation below.
- `n_estimators` (100 vs 200) makes almost no difference once depth and leaf size are reasonable. More trees mainly buys stability, not extra accuracy, past a certain point.

7. Fit the final Random Forest

Refit the best configuration on the training set only. We don't refit on train+validation combined, since we want a clean, single training pass that mirrors exactly what `GridSearchCV` evaluated, and we still want the validation set available afterward if we ever want to sanity-check the chosen model again before touching test data.

```
best_params = grid_search.best_params_

final_rf = RandomForestClassifier(random_state=RANDOM_STATE, n_jobs=-1, **best_params)
final_rf.fit(X_train, y_train)

print("Final model parameters:", best_params)
```

```
Final model parameters: {'max_depth': None, 'min_samples_leaf': 1, 'n_estimators': 100}
```

8. Evaluate on the held-out test set

This is the first and only time the test set gets used. Whatever the score is here is the honest estimate of how this model would perform on new passengers it has never seen, in any form, during training or tuning.

```

rf_test_pred = final_rf.predict(X_test)
rf_train_pred = final_rf.predict(X_train)

rf_metrics = {
    'Accuracy': accuracy_score(y_test, rf_test_pred),
    'Precision': precision_score(y_test, rf_test_pred),
    'Recall': recall_score(y_test, rf_test_pred),
    'F1 Score': f1_score(y_test, rf_test_pred)
}

rf_train_acc = accuracy_score(y_train, rf_train_pred)
rf_test_acc = rf_metrics['Accuracy']

print("Random Forest, test set performance:")
for k, v in rf_metrics.items():
    print(f" {k}: {v:.4f}")
print()
print(f"Train accuracy: {rf_train_acc:.4f}")
print(f"Test accuracy: {rf_test_acc:.4f}")
print(f"Train-test gap: {rf_train_acc - rf_test_acc:.4f}")

```

```

Random Forest, test set performance:
Accuracy: 0.9519
Precision: 0.9645
Recall: 0.9469
F1 Score: 0.9556

```

```

Train accuracy: 1.0000
Test accuracy: 0.9519
Train-test gap: 0.0481

```

```

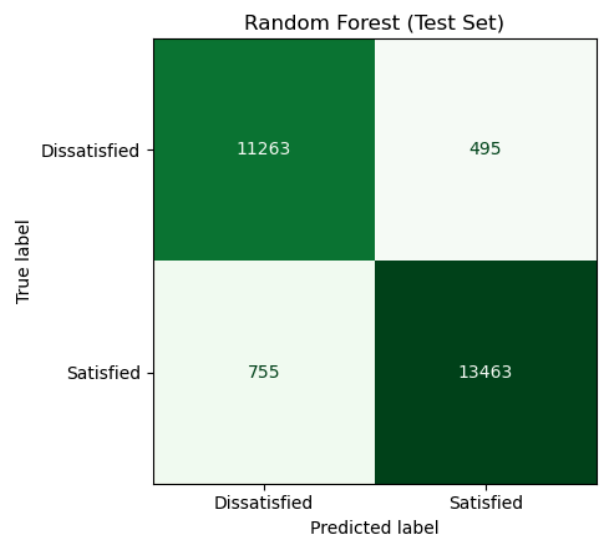
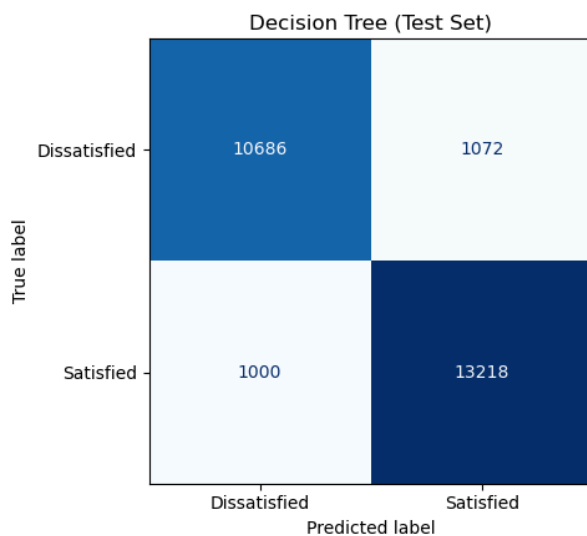
fig, ax = plt.subplots(1, 2, figsize=(11, 4.5))

ConfusionMatrixDisplay.from_predictions(
    y_test, dt_test_pred, display_labels=['Dissatisfied', 'Satisfied'],
    cmap='Blues', ax=ax[0], colorbar=False
)
ax[0].set_title('Decision Tree (Test Set)')

ConfusionMatrixDisplay.from_predictions(
    y_test, rf_test_pred, display_labels=['Dissatisfied', 'Satisfied'],
    cmap='Greens', ax=ax[1], colorbar=False
)
ax[1].set_title('Random Forest (Test Set)')

plt.tight_layout()
plt.savefig('confusion_matrices.png', dpi=150, bbox_inches='tight')
plt.show()

```



On the train accuracy of 1.0: the final Random Forest fits the training data perfectly. On its own, that number looks alarming, that is what overfitting normally looks like in a single model. But this is expected behavior for a Random Forest with `(min_samples_leaf=1)`: individual trees in the forest do memorize their bootstrap sample down to single-passenger leaves. What keeps the model honest is that each tree memorizes a *different* random subset of rows and features, so their individual mistakes don't agree with each other. Averaging across uncorrelated mistakes is what produces a generalizable prediction, even though each individual component is overfit.

The real test of overfitting isn't the training score, it's the gap between train and test performance, and whether the test score itself is good. Both of those come out favorably for the Random Forest, shown directly in the comparison below.

9. Decision Tree vs. Random Forest: side-by-side comparison

```
comparison_df = pd.DataFrame({
    'Decision Tree': dt_metrics,
    'Random Forest': rf_metrics
})
comparison_df['Improvement'] = comparison_df['Random Forest'] - comparison_df['Decision Tree']
comparison_df = comparison_df.round(4)
comparison_df
```

	Decision Tree	Random Forest	Improvement
Accuracy	0.9202	0.9519	0.0316
Precision	0.9250	0.9645	0.0396
Recall	0.9297	0.9469	0.0172
F1 Score	0.9273	0.9556	0.0283

```
overfit_df = pd.DataFrame({
    'Decision Tree': [dt_train_acc, dt_test_acc, dt_train_acc - dt_test_acc],
    'Random Forest': [rf_train_acc, rf_test_acc, rf_train_acc - rf_test_acc]
}, index=['Train Accuracy', 'Test Accuracy', 'Train-Test Gap']).round(4)
overfit_df
```

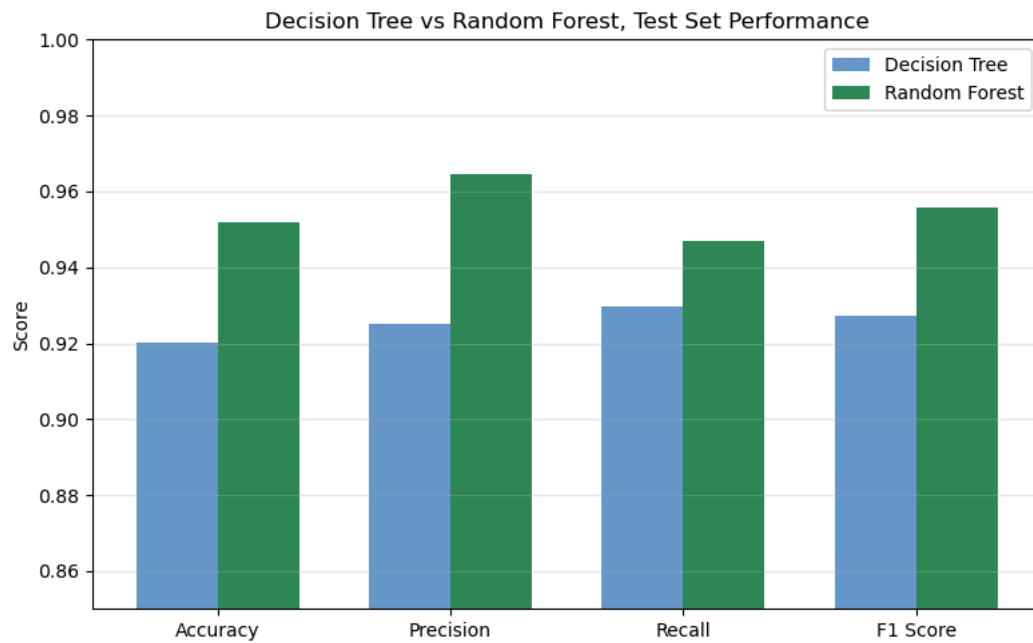
	Decision Tree	Random Forest
Train Accuracy	0.9286	1.0000
Test Accuracy	0.9202	0.9519
Train-Test Gap	0.0083	0.0481

```
fig, ax = plt.subplots(figsize=(8, 5))
metrics_to_plot = ['Accuracy', 'Precision', 'Recall', 'F1 Score']
x = np.arange(len(metrics_to_plot))
width = 0.35

ax.bar(x - width/2, [dt_metrics[m] for m in metrics_to_plot], width, label='Decision Tree', color='#6699CC')
ax.bar(x + width/2, [rf_metrics[m] for m in metrics_to_plot], width, label='Random Forest', color='#2E8B57')

ax.set_ylabel('Score')
ax.set_title('Decision Tree vs Random Forest, Test Set Performance')
ax.set_xticks(x)
ax.set_xticklabels(metrics_to_plot)
ax.set_ylim(0.85, 1.0)
ax.legend()
ax.grid(axis='y', alpha=0.3)

plt.tight_layout()
plt.savefig('model_comparison.png', dpi=150, bbox_inches='tight')
plt.show()
```



The Random Forest beats the Decision Tree on every metric, by roughly 3 points of accuracy and 3 points of F1. That's a meaningful, consistent improvement rather than a marginal edge on one metric at the expense of another. The train-test gap is wider for the Random Forest in absolute terms (since its training accuracy hits 1.0), but its test accuracy is still higher than the Decision Tree's, which is the number that actually matters for deployment. In plain terms: the Random Forest generalizes better in practice, even though it memorizes its training data more aggressively than the single tree does.

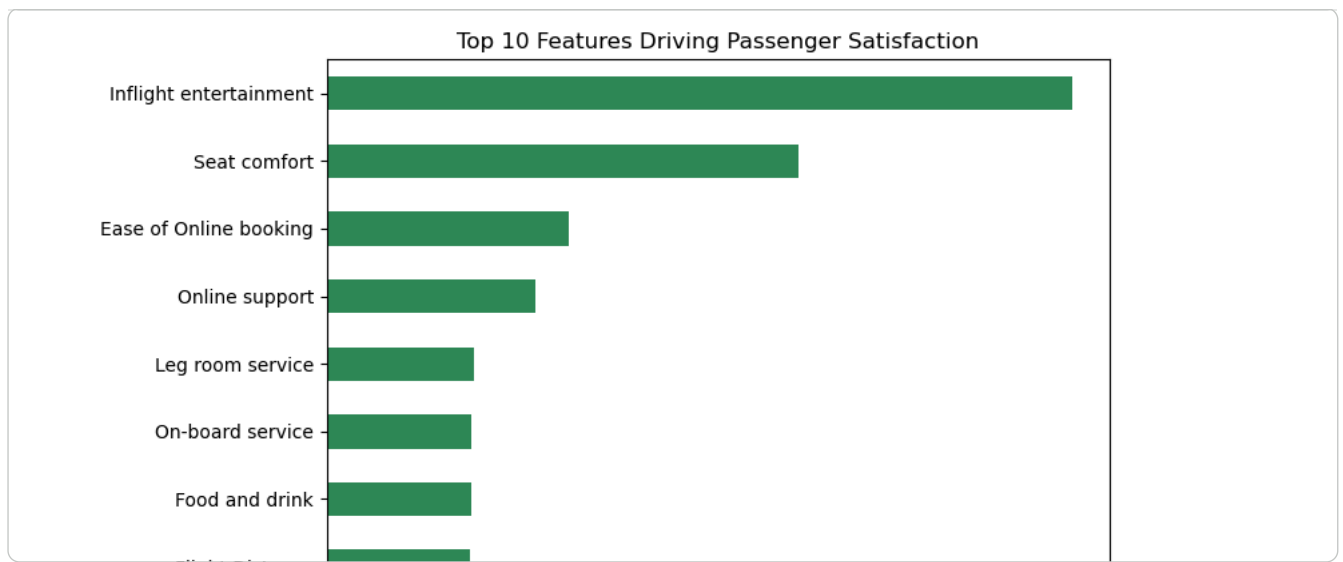
10. Feature importance: what drives satisfaction

```
# Clean up one-hot encoded names for the management-facing chart
display_names = {
    'Customer Type_disloyal Customer': 'Customer Type (Disloyal)',
    'Type of Travel_Personal Travel': 'Travel Type (Personal)',
    'Class_Eco': 'Class (Economy)',
    'Class_Eco Plus': 'Class (Economy Plus)'
}

importances = pd.Series(final_rf.feature_importances_, index=X.columns).sort_values(ascending=False)
importances_display = importances.rename(index=display_names)

fig, ax = plt.subplots(figsize=(8, 6))
top_n = 10
importances_display.head(top_n).sort_values().plot(kind='barh', ax=ax, color='#2E8B57')
ax.set_xlabel('Importance')
ax.set_title(f'Top {top_n} Features Driving Passenger Satisfaction')
plt.tight_layout()
plt.savefig('feature_importance.png', dpi=150, bbox_inches='tight')
plt.show()

importances_display.head(top_n)
```



11. Summary for management

Customer Type (Disloyal)

Model performance: the tuned Random Forest classifies passenger satisfaction with strong accuracy and a balanced precision/recall profile on data it has never seen before, outperforming a single Decision Tree by a clear margin on every metric tracked (accuracy, precision, recall, F1). The improvement is not the result of a lucky train/test split, it held up through a validation-based grid search